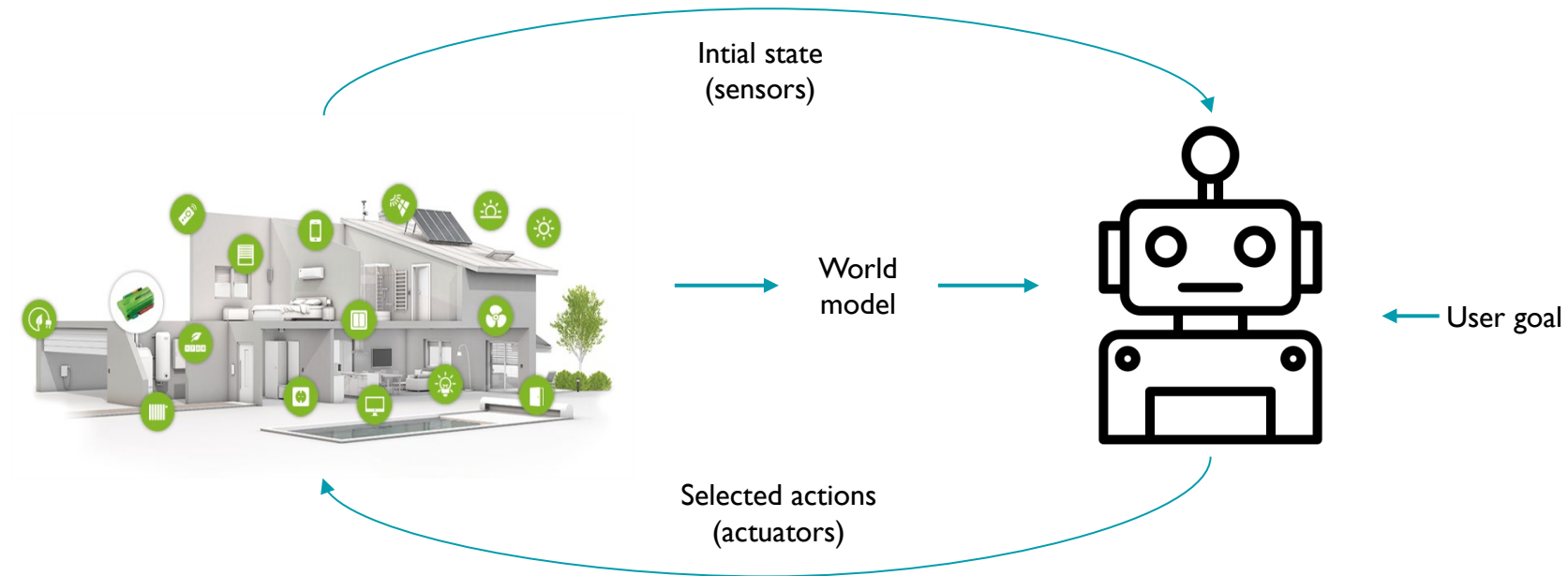


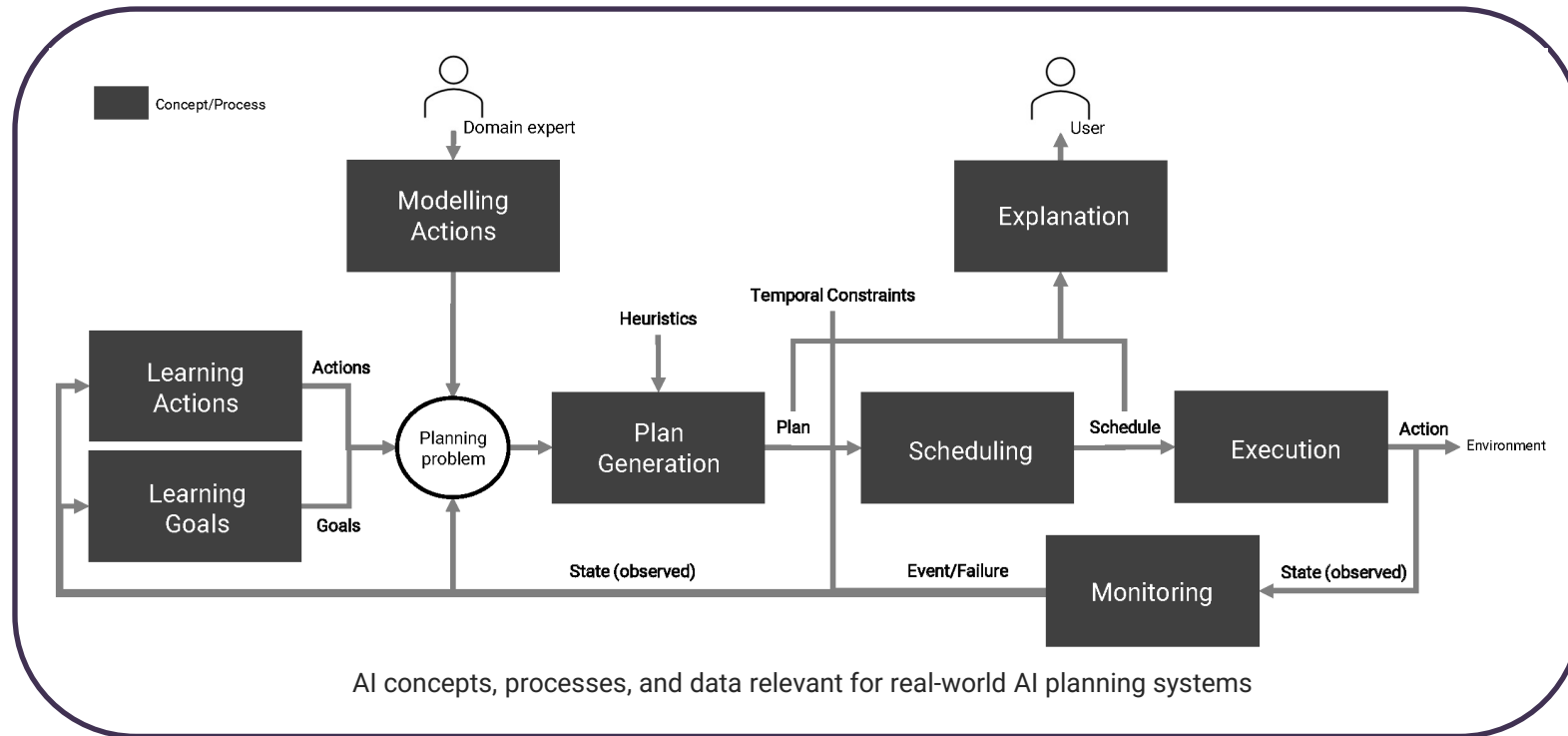
University of Stuttgart
Germany

AI Planning Tools

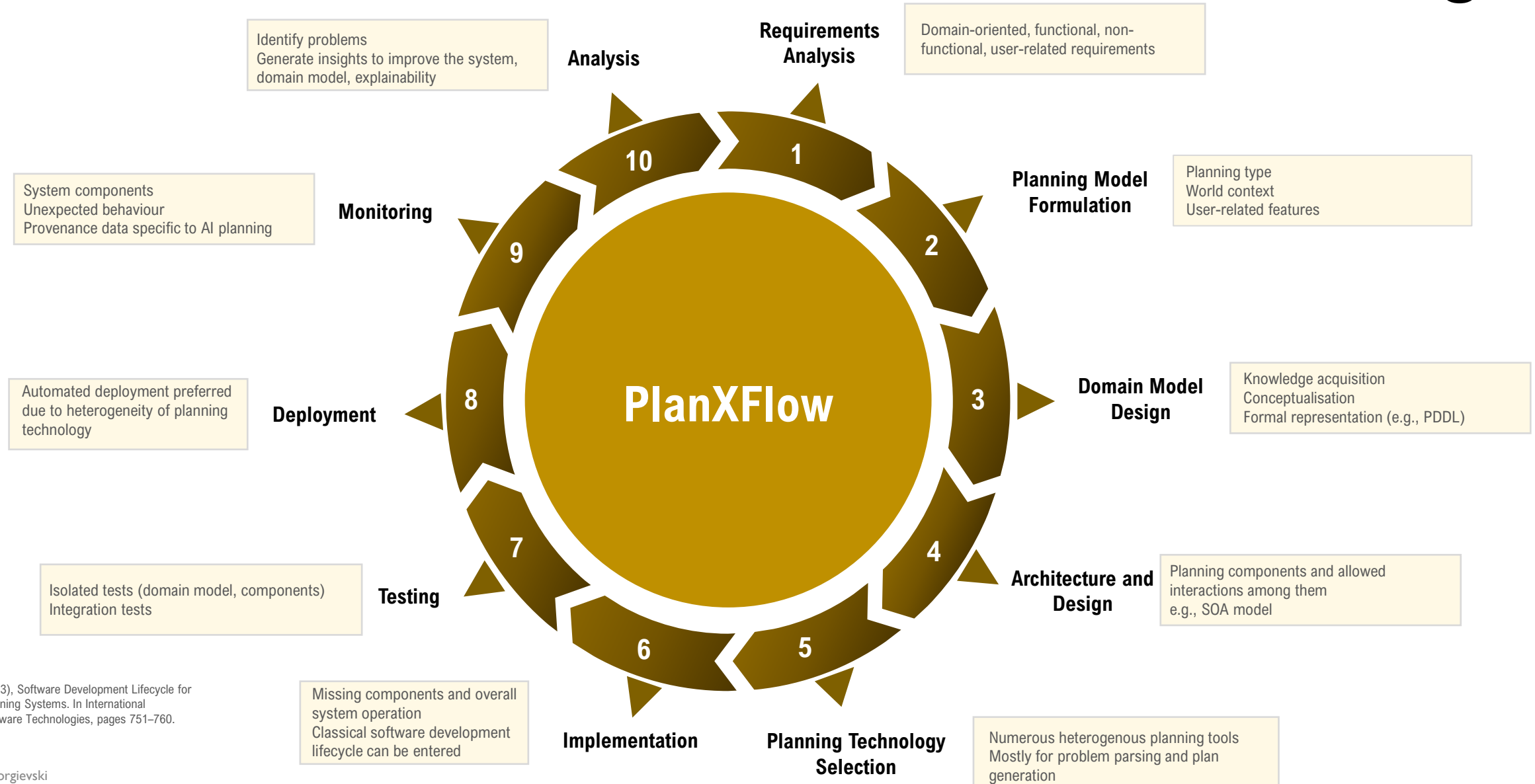
From a domain model to action execution



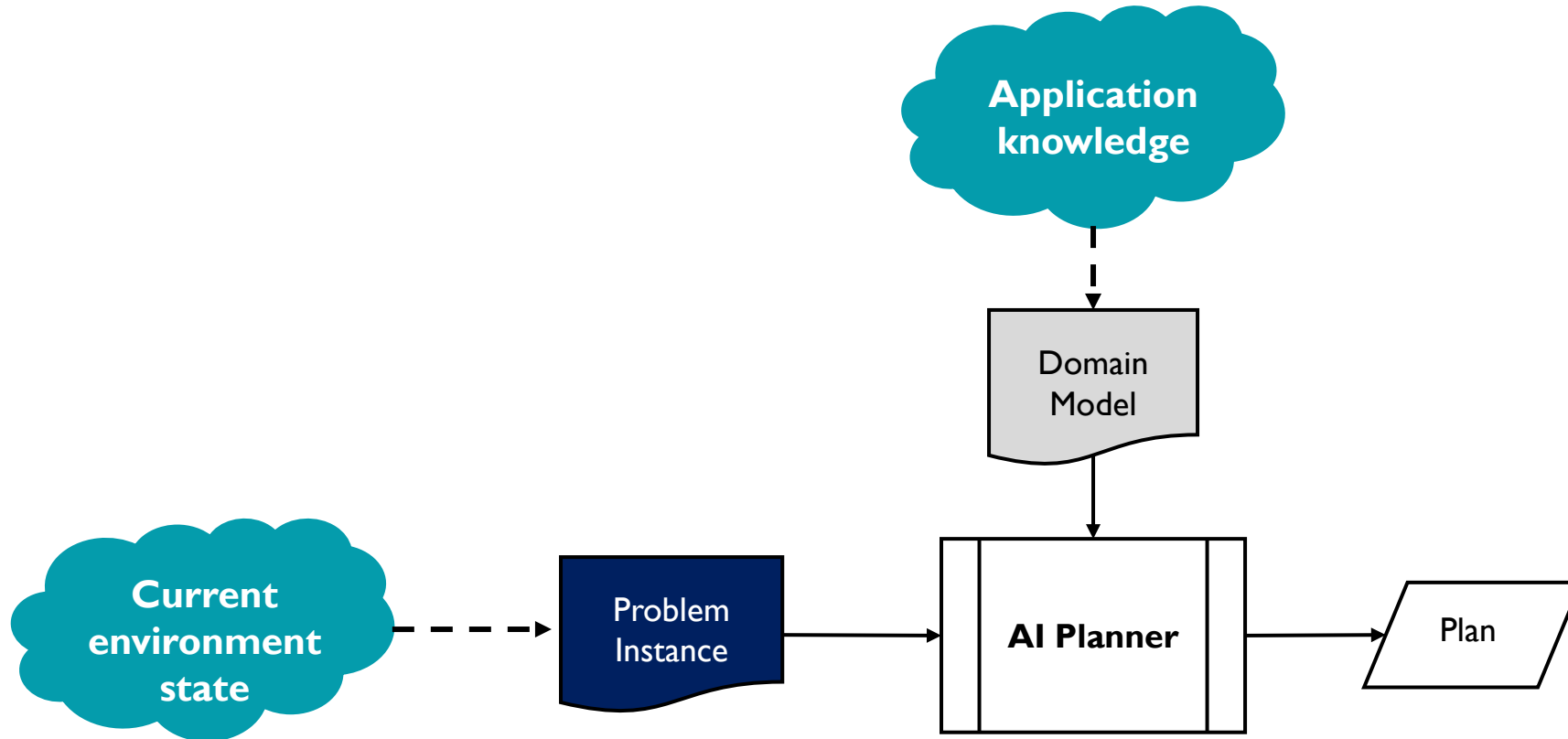
General Planning Framework



SDLC for AI Planning



Key Components



Two artefacts, two lifetimes



Domain, written once

The vocabulary and physics of the world: types, predicates, and what each action requires and changes.

Reused across every situation.

Changes only when the world changes.



Problem instance, rebuilt continuously

A snapshot: which objects exist, what is true now, and what we want to achieve right now.

Generated from sensors + the static database.

Different on every cycle.

Stage I

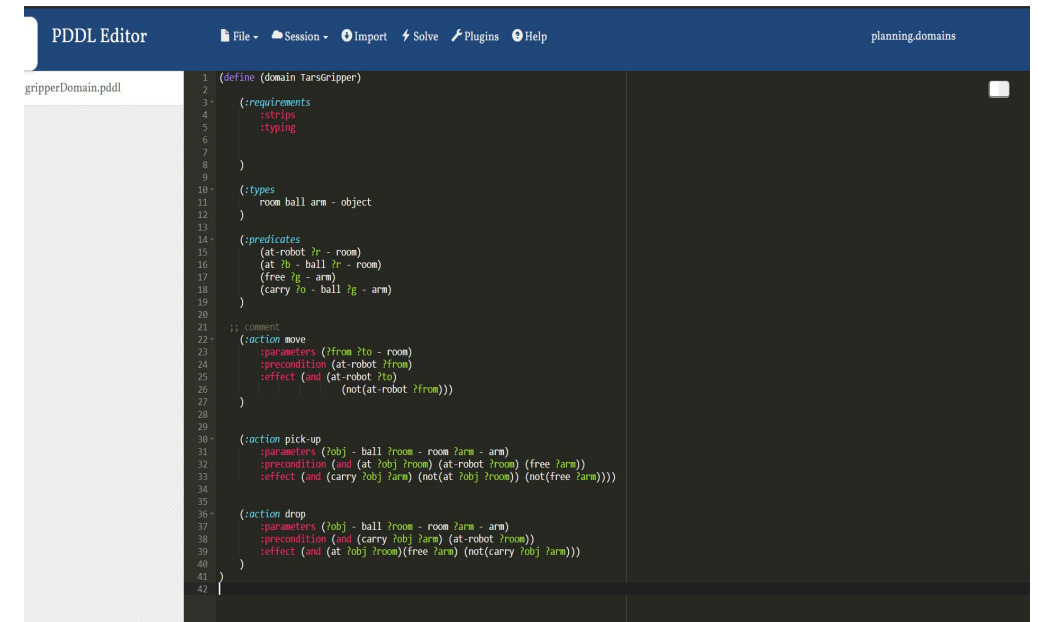
MODEL THE DOMAIN

Options

- Human-authored domain models
 - Any text editor
 - PDDL Editor – Planning.Domains
 - PDDL Extension for Visual Studio (VS)
 - itSIMPLE
- Assisted domain model generation
 - LLMs

PDDL Editor – Planning.Domains

- Write PDDL in the browser
 - Syntax highlighting, bracket matching, auto-completion, code folding
- Test without local setup
 - Use hosted planners directly to test a domain model in the same window
- Access: <https://editor.planning.domains/>



```
1 (define (domain Tarsgrripper)
2
3   (:requirements
4     :strips
5     :typing
6   )
7
8   (:types
9     room ball arm - object
10  )
11
12  (:predicates
13    (at-robot ?r - room)
14    (at ?b - ball ?r - room)
15    (free ?g - arm)
16    (carry ?a - ball ?g - arm)
17  )
18
19  ;; comment
20
21  (:action move
22    :parameters (?from ?to - room)
23    :precondition (at-robot ?from)
24    :effect (and (at-robot ?to)
25               (not(at-robot ?from)))
26  )
27
28
29
30
31  (:action pick-up
32    :parameters (?obj - ball ?room - room ?arm - arm)
33    :precondition (and (at ?obj ?room) (at-robot ?room) (free ?arm))
34    :effect (and (carry ?obj ?arm) (not(at ?obj ?room)) (not(free ?arm)))
35  )
36
37  (:action drop
38    :parameters (?obj - ball ?room - room ?arm - arm)
39    :precondition (and (carry ?obj ?arm) (at-robot ?room))
40    :effect (and (at ?obj ?room) (free ?arm) (not(carry ?obj ?arm)))
41  )
42 )
```

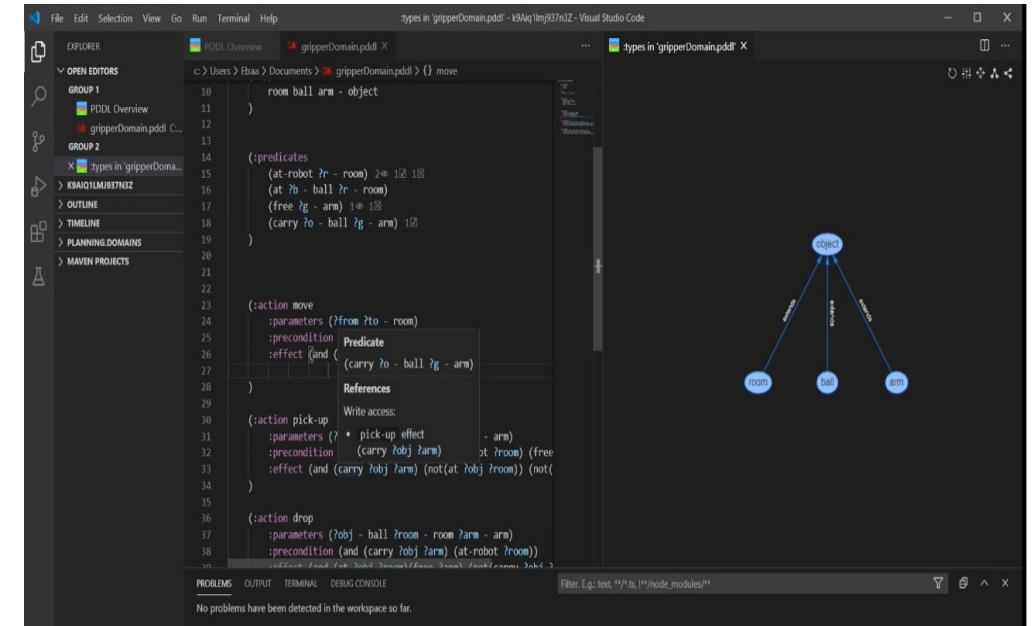
Thanks to Ebaa Alnazer

When to use it

- Learn PDDL modelling
- First drafts and quick experiments

PDDL extension for VS

- Write PDDL in IDE
 - Syntax highlighting, auto-completion
- Parser and validator (VAL) built in
 - Catches syntax and many semantic errors while typing, with line-level guidance
- Visualise a domain model
 - See objects, types and the structure of a problem instead of reading raw text
- Access:
<https://marketplace.visualstudio.com/items?itemName=jan-dolejsi.pddl>



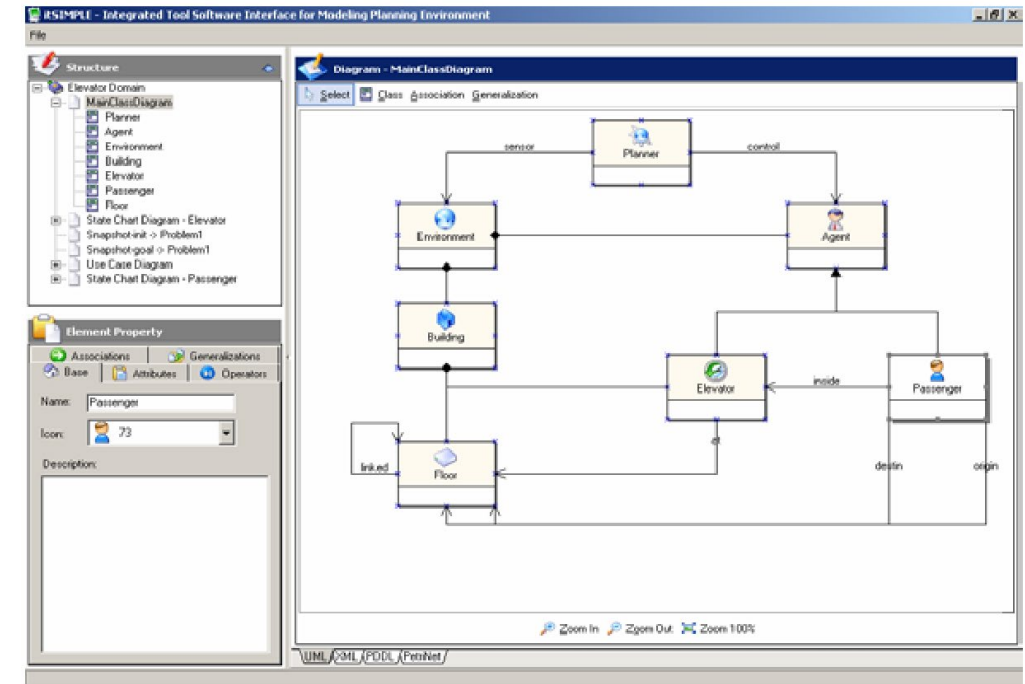
Thanks to Ebba Alnazer

When to use it

- Learn PDDL modelling
- Larger domain models
- When validation needed

itSIMPLE

- Model in UML
 - Describe a domain as an object-oriented UML model
- Export to PDDL
 - Translate an UML model into PDDL domain model
- Validate via Petri nets
 - Convert the model to Petri nets to analyse its static and dynamic behaviour and surface inconsistencies before plan generation
- Analyse returned plans
 - Visual, animated inspection of plan execution (how state and variables evolve) to help debug the domain model
- Access: <https://code.google.com/archive/p/itsimple/>



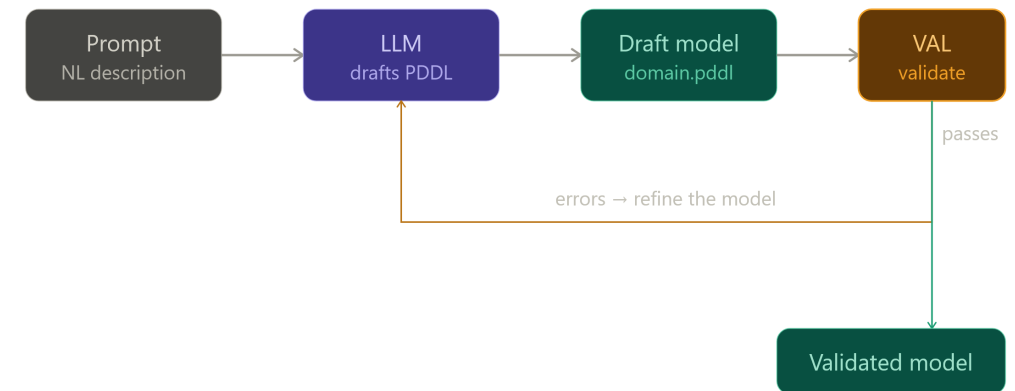
Vaquero et al (2006), itSIMPLE: An Integrated Tool for Modeling and Analyzing Planning Domains, ICAPS.

When to use it

- Early, requirements-to-model design phase
- Larger or real-world domains where structure matter
- When domain visualization needed

LLM-Assisted Domain Modelling

- Let an LLM construct a domain model
 - The LLM drafts a domain model from a natural language (NL) description
 - A validator (e.g., VAL) used to validate the drafted domain model
 - Validation result used as a feedback to LLM to improve the domain model
- Possible issues
 - Mismatched types
 - Invented predicates
 - Missing preconditions/effects
 - Preconditions too strong or too weak
- Tools
 - Any LLMs in combination with VAL or an AI planner
 - L2P - LLM-driven Planning Model library kit: <https://github.com/AI-Planning/l2p>



When to use it

- Cold start; sketching a first draft from a NL description
- Unfamiliar domain where you would struggle to name predicates/actions/tasks yourself
- Note it is weakest at finishing a domain model (use it always with a validator/planner in the loop)

Domain Model Debugging

- Use a validator or a planner
 - A validator checks a plan against a domain model and reports actions whose preconditions fail
 - A planner computes either finds a plan or does not (even though a plan is expected)
- No plan found
 - A planner searched but nothing found
 - Usually the domain model is too restrictive, not the task impossible:
 - a precondition is stronger than it should be
 - an action's effect never establishes the goal predicate
 - the goal uses a predicate nothing produces
- A plan found, but a wrong one
 - A planner finds a solution, but the plan does not make sense:
 - a missing delete-effect let an old fact persist
 - a precondition is too weak, so an illegal action fired
 - run VAL to see exactly which step diverged

Stage 2

GENERATE THE PROBLEM INSTANCE

A problem instance has three parts

- **:objects**
 - **What:** which objects exist
 - **Where:** typically from static / configuration data
- **:init**
 - **What:** what is true now
 - **Where:** from live sensor readings, current system state, and configuration data (device and topology databases), abstracted into predicates that are defined in the domain model
- **:goal**
 - **What:** what we want now
 - **Where:** from a current task, request, or policy, compiled into a goal condition using predicates from the domain model

A mapping example

- **:objects**
 - Static from a database: *office1* is a room, *light1* is a light, *hvac1* is HVAC
- **:init**
 - Static from a database: *light1* is in *office1*
 - Live from sensors/context: *office1* is occupied, *light1* is off, illuminance in *office1* is low
- **:goal**
 - Building policy as a goal: occupied rooms must be adequately lit and at comfort temperature

```
(define (problem office-cycle-07)
  (:domain smart-office)

  (:objects
    office1 - room
    light1  - light
    hvac1   - hvac)

  (:init
    (occupied office1)
    (in light1 office1)
    (illuminance-low office1))

  (:goal (and
    (adequately-lit office1)
    (comfort-temp office1))))
```


Automated problem instance generation

generate_problem

```
domain ← load_domain()

on event (state change | new goal | failure):

    state ← snapshot_sensors()
    config ← read_configuration()

    objects ← to_objects(config)
    init ← topology(config) ∪ sensed(state)
    goal ← compile_goal(task)

    problem ← fill_template(domain.name, objects, init, goal)

    if not valid(problem, domain):
        repair_or_abort()

return problem
```

fill_template

```
function fill_template(domain_name, objs, init,
goal):

    return "
(define (problem <name>)
  (:domain domain_name)
  (:objects {objs})
  (:init    {init})
  (:goal    {goal}))
"
```

Stage 3

CHOOSE A PLANNER

Planning objectives

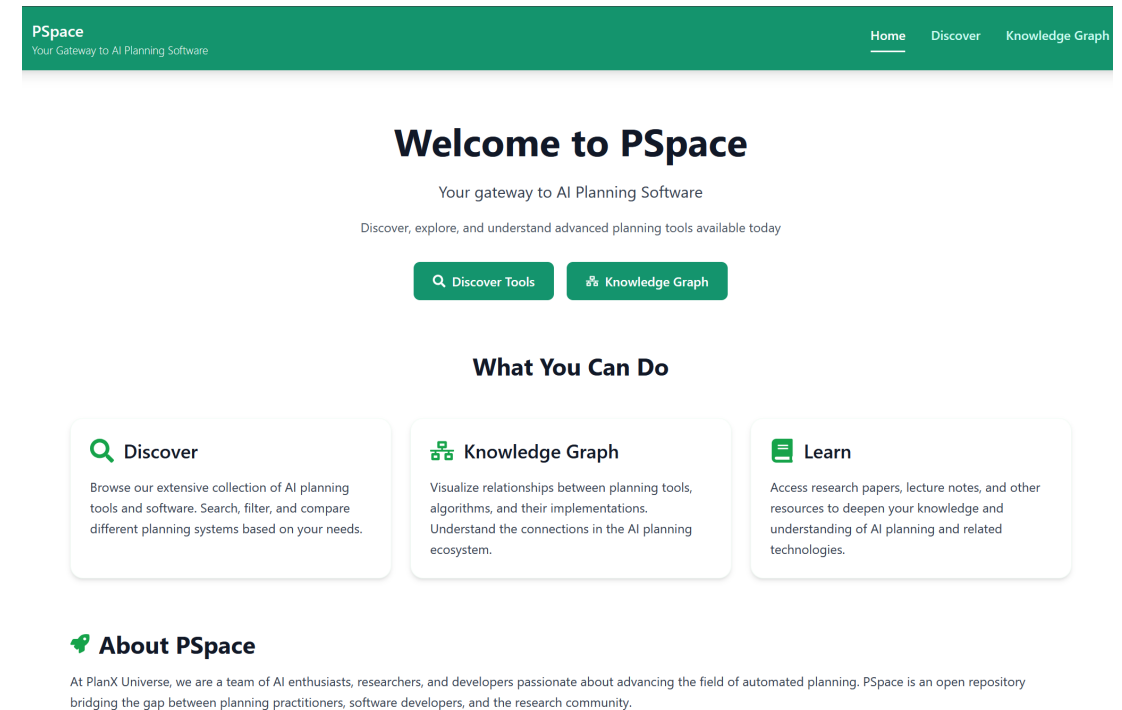
- Feasible planning: find any plan that achieves the goal
- Cost-optimal planning: find a plan that minimises total action cost
- Satisficing planning: find a plan, prefer lower-cost
- Bounded planning: find a plan whose cost is within a given bound
- Top-k planning: find k plans such that no lower-cost plans exist
- Risk-sensitive planning: find a plan that optimises expected utility under a given risk model
- Anytime planning: quickly find a plan and improve it over time
- Agile planning: find a plan within short computation time

How to choose

- Depends on the expressiveness of the domain model and problem instances
- **Do actions take time?**
 - Yes → Temporal planner
 - No → Classical planner
- **Quantities (e.g., energy)?**
 - Yes → Numeric planner
 - No → Classical planner
- **Cheapest plan?**
 - Yes → Cost-optimal planner
 - No → Agile or satisficing planner
- **Does the application domain have recipe-like knowledge?**
 - Yes → HTN planner
 - No → Classical planner

Where to look

- Some standalone state-of-the-art AI planners
 - Classical, cost-optimal, satisficing planners:
 - Fast Downward: <https://www.fast-downward.org/latest/>
 - Fast Forward: <https://fai.cs.uni-saarland.de/hoffmann/ff.html>
 - Numeric/temporal planners:
 - Metric-FF: <https://fai.cs.uni-saarland.de/hoffmann/metric-ff.html>
 - OPTIC: <https://github.com/roveri-marco/optic>
 - Fast HTN planners
 - SH: <https://planx-universe.github.io/sh/>
 - Cost-optimal HTN planners
 - PANDA: <https://github.com/panda-planner-dev>
- Planning as a Service: <https://solver.planning.domains/>
- Searchable registry of planners: <https://pspace.planxverse.com/>

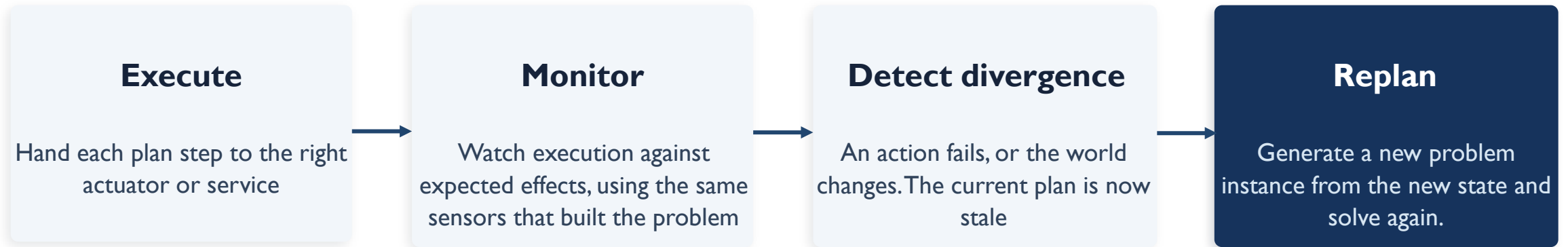


PSpace being developed by our department

Stage 4

EXECUTE A PLAN

Execute, monitor, replan



With this loop, planning stops is not a one-shot call but a control strategy
(the whole point in a smart environment)

Recap

- Model the application domain once; PDDL extension for VS is a good choice
- Generate problem instances automatically from sensors + static information, on each environment event
- Validate a domain model and problem instances; silent bad input is the usual failure
- Choose a planner by capability and availability (PSpace)
- Close the loop: execute, monitor, and replan when the world changes

